

*Inteligência Artificial
e Computacional*

ELT574

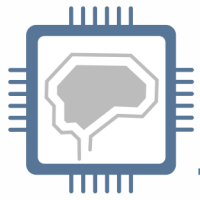
Aprendizado de Máquinas

Redes perceptron
multicamadas (PMC)

Conteudista: Rodolpho Neves

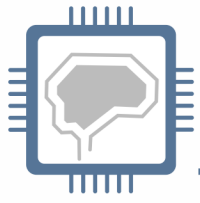
UFV
Universidade Federal de Viçosa

cead_{UFV}
Coordenadoria de
Educação Aberta e a Distância

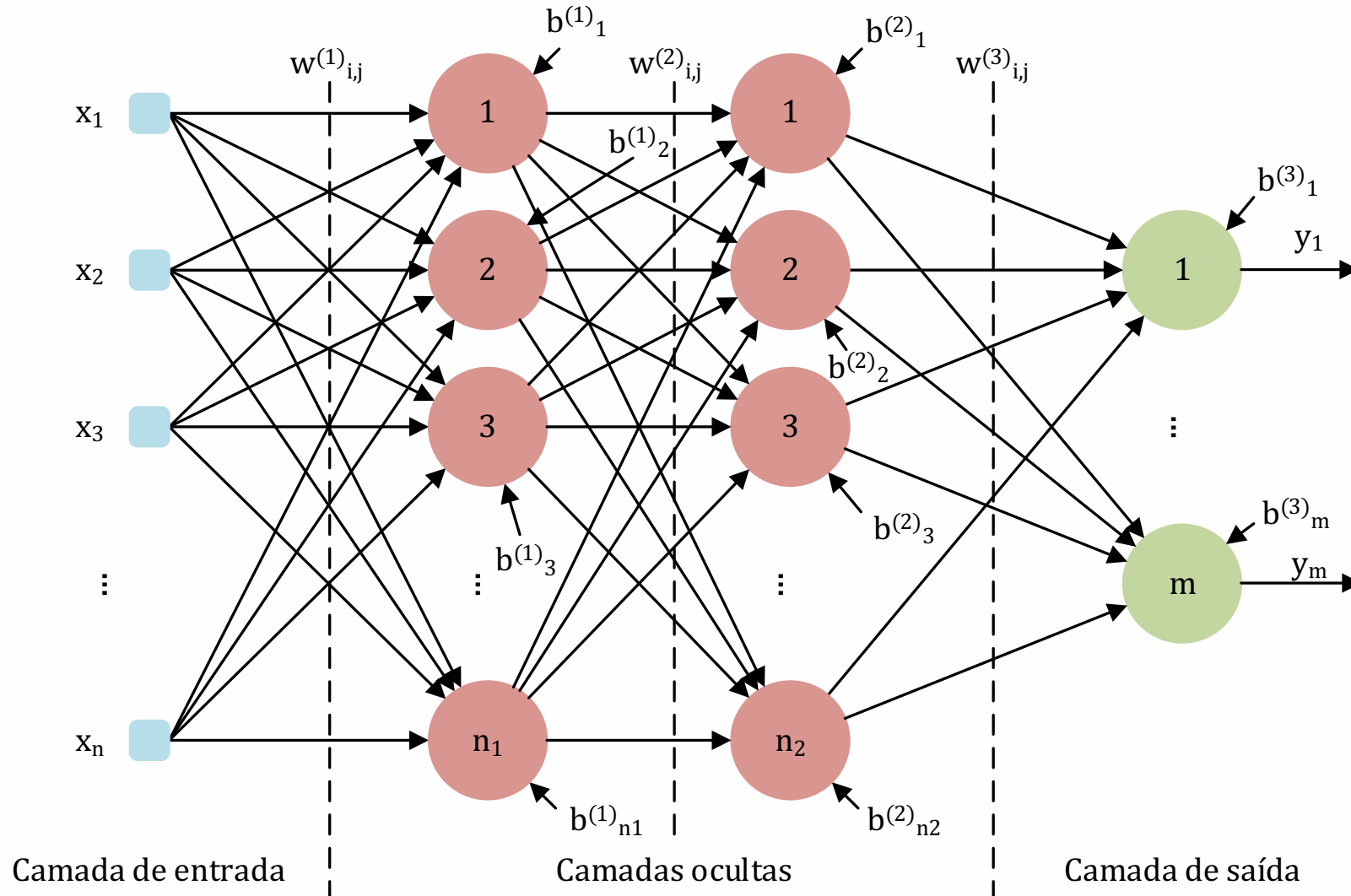


Problemas não lineares

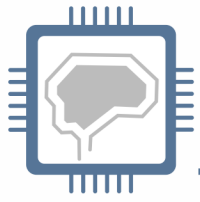
- Problemas de classificação e estimação
 - Topologias lineares
 - Perceptron e Adaline
- Problemas com topologias mais complexas (não lineares)
- Rede Perceptron Multicamadas (PMC)
 - Elementos lineares
 - Ferramentas de otimização
 - Compõem estruturas complexas
- Problema é dividido em várias partes lineares
- Resolvido por cada perceptron da PMC



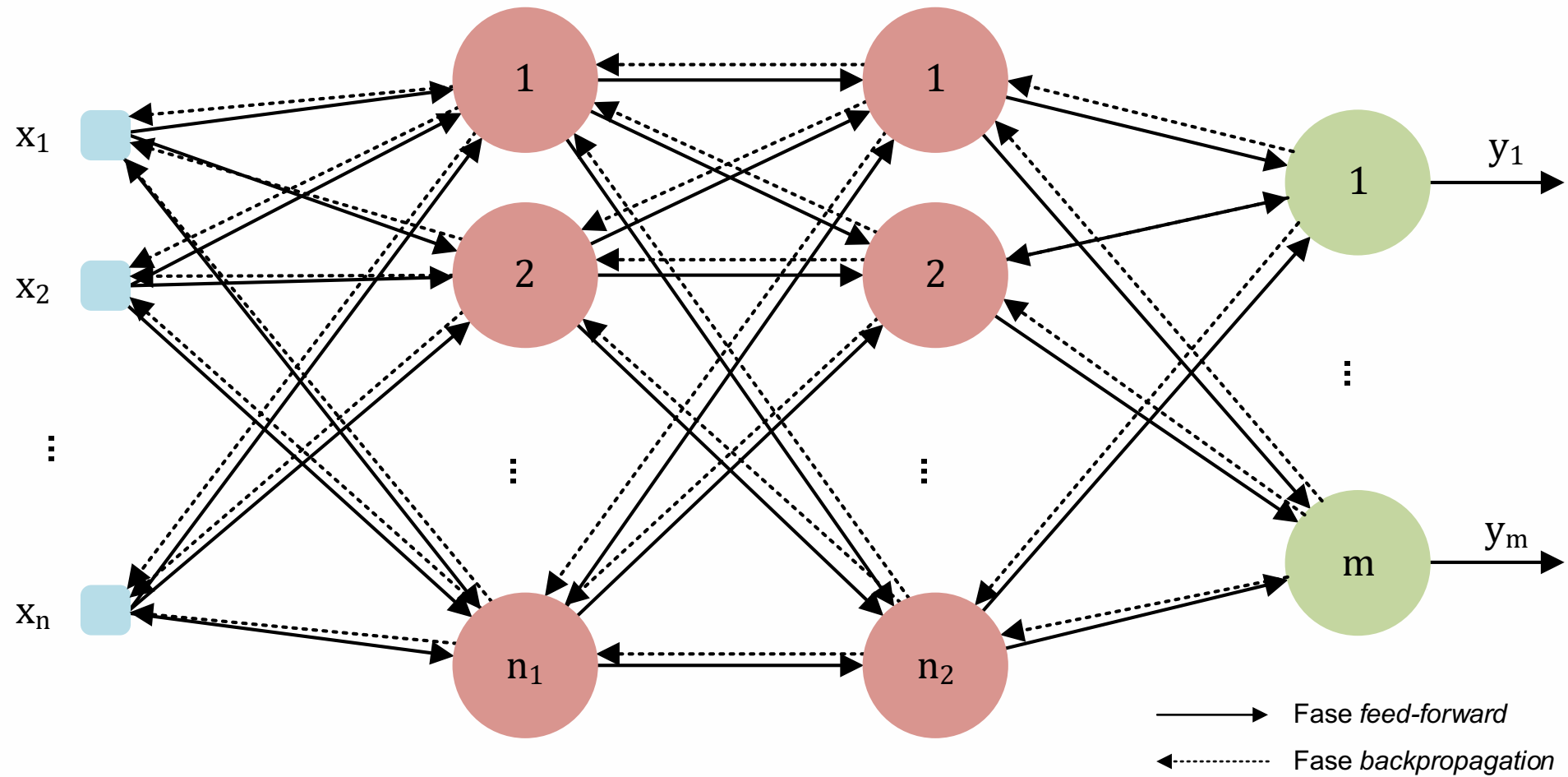
Perceptron multicamadas (PMC)



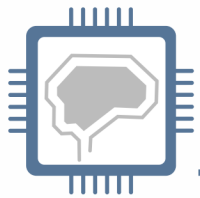
Estrutura de uma rede perceptron multicamadas.



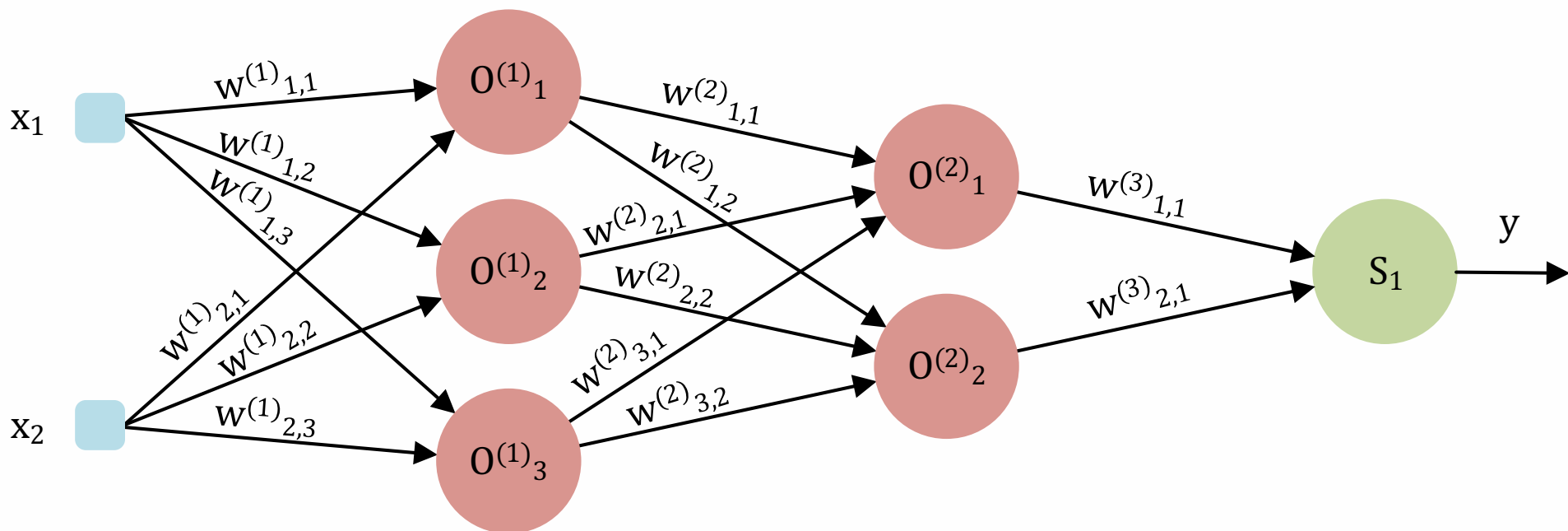
Algoritmo *backpropagation*



Fluxo de funcionamento das fases *feed-forward* e *backpropagation*.



Aplicação do *backpropagation*

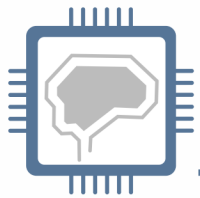


Exemplo de aplicação do *backpropagation*.

$$I_1^{(1)} = XW_j^{(1)} = [x_1, x_2] \begin{bmatrix} w_{1,1}^{(1)} \\ w_{2,1}^{(1)} \end{bmatrix} = x_1 w_{1,1}^{(1)} + x_2 w_{2,1}^{(1)}$$

$$y_1^{(3)} = g(I_1^{(3)}) = g(y_1^{(2)} w_{1,1}^{(3)} + y_2^{(2)} w_{2,1}^{(3)})$$

$$I_2^{(2)} = Y^{(L-1)}W_j^{(L)} = [y_1^{(1)}, y_2^{(1)}, y_3^{(1)}] \begin{bmatrix} w_{1,2}^{(2)} \\ w_{2,2}^{(2)} \\ w_{3,2}^{(2)} \end{bmatrix} = y_1^{(1)} w_{1,2}^{(2)} + y_2^{(1)} w_{2,2}^{(2)} + y_3^{(1)} w_{3,2}^{(2)}$$



Aplicação do *backpropagation*

- Erro médio quadrático:

$$E(k) = \frac{1}{2} \sum_{j=1}^m \left(d_j(k) - y_j^{(Lm)} \right)^2$$

- Erro médio pelo total de amostras:

$$EQM = \frac{1}{p} \sum_{k=1}^p E(k)$$

- Atualizando os pesos da última camada:

$$W_j^{(Lm)}(t+1) = W_j^{(Lm)}(t) + \eta \delta_j^{(Lm)} y_j^{(Lm-1)}$$

$$\delta_j^{(Lm)} = \left(d_j - y_j^{(Lm)} \right) g' \left(I_j^{(Lm)} \right)$$

- Atualizando os pesos das camadas intermediárias:

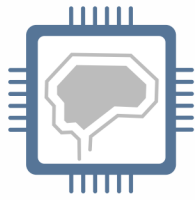
$$W_j^{(L)}(t+1) = W_j^{(L)}(t) + \eta \delta_j^{(L)} y_j^{(L-1)}$$

$$\delta_j^{(L)} = \delta_j^{(Lm)} W_j^{(Lm)} g' \left(I_j^{(L)} \right)$$

- Atualizando os pesos da primeira camada:

$$W_j^{(1)}(t+1) = W_j^{(1)}(t) + \eta \delta_j^{(1)} x_i$$

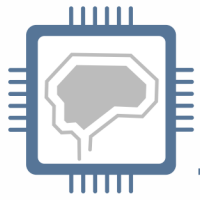
$$\delta_j^{(1)} = \delta_j^{(2)} W_j^{(2)} g' \left(I_j^{(1)} \right)$$



Algoritmo de treinamento do PMC

Início – Fase de treinamento – Rede Perceptron Multicamada:

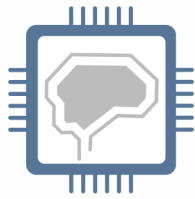
1. Carregar o conjunto de treinamento para a matriz X com a entrada do limiar -1 ;
2. Carregar o conjunto de saídas desejadas para o vetor d ;
3. Atribuir valores aleatórios e normalizados aos pesos sinápticos W (com o w_0) de todas as camadas da rede;
4. Escolher o valor da taxa de aprendizado η e para o limiar de precisão ε ;
5. Determine o número máximo de épocas de treinamento $epoca_{max}$;
6. Iniciar o contador de épocas do treinamento ($epoca = 0$);



Algoritmo de treinamento do PMC

7. Faça até que $|EQM^{atual} - EQM^{anterior}| \leq \varepsilon$:
 1. Faça $EQM^{anterior} = EQM^{atual}$;
 2. Para todas as amostras de treinamento $\{X^{(k)}, d^{(k)}\}$, faça:
 1. Calcular o valor de $I_j^{(1)}$ e $y_j^{(1)}$ para a primeira camada;
 2. Calcular o valor de $I_j^{(L)}$ e $y_j^{(L)}$ para todas as camadas intermediárias;
 3. Calcular o valor de $I_j^{(Lm)}$ e $y_j^{(Lm)}$ para a última camada;
 4. Calcular o valor de $\delta_j^{(Lm)}$ para a última camada;
 5. Atualizar o valor de $W_j^{(Lm)}$ para a última camada;
 6. Calcular o valor de $\delta_j^{(L)}$ e atualizar $W_j^{(L)}$ para as camadas intermediárias;
 7. Calcular o valor de $\delta_j^{(1)}$ para a primeira camada;
 8. Atualizar o valor de $W_j^{(1)}$ para a primeira camada;
 3. Calcular o valor de $y_j^{(Lm)}$;
 4. Calcular o erro EQM^{atual} ;
 5. Faça $epoca = epoca + 1$;
 6. Se $epoca > epoca_{max}$, pare o treinamento.

Fim

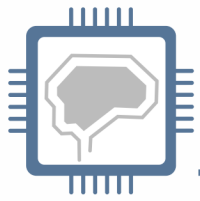


Algoritmo de operação do PMC

Início – Fase de operação – Rede Perceptron Multicamada:

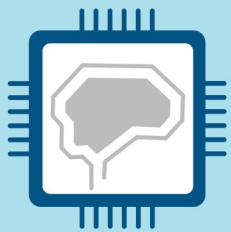
1. Carregar o vetor de amostra X ;
2. Concatenar o valor -1 à matriz X , representando a entrada para o *bias*;
3. Carregar o vetor de pesos resultante do treinamento W de todas os neurônios e todas as camadas;
4. Calcular o valor de $I_j^{(1)}$ e $y_j^{(1)}$ para a primeira camada;
5. Calcular o valor de $I_j^{(L)}$ e $y_j^{(L)}$ para todas as camadas intermediárias;
6. Calcular o valor de $I_j^{(Lm)}$ e $y_j^{(Lm)}$ para a última camada;
7. Disponibilizar a saída da rede de acordo com os valores de $y_j^{(Lm)}$;

Fim



Considerações finais

- Algoritmo de backpropagation
 - Realiza atualizações nos parâmetros
 - Dependendo do tamanho do banco de dados e da complexidade da rede
 - Pode ser lento e custoso computacionalmente
- Outras configurações de backpropagation e métodos de atualização de parâmetros
 - Momentum
 - Levenberg-Marquardt
 - Gradiente descendente estocástico
- Perceptron multicamadas (PMC)
 - Combina funções dos elementos mais simples
 - Treinamento pode necessitar de refinamento para ser eficiente e com boa generalização



Inteligência Artificial e Computacional

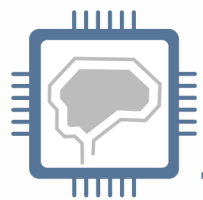
ELT574

Aprendizado de Máquinas

Conteudista:

Rodolpho Neves

rodolpho.neves@ufv.br



Realização

UFV

Universidade Federal de Viçosa

